

Rappel théorique

Rappel théorique

Types de variables

Une variable a un nom et un type. Le type fixe la place en mémoire, la façon d'encoder la valeur, et les opérations permises. On la déclare avant de l'utiliser : `type nom;` ou `type nom = valeur;`.

Type	Contenu
<code>char</code>	caractère, ou entier sur 8 bits
<code>int</code>	entier. Taille selon la machine ; souvent 32 bits sur un PC actuel, de -2^{31} à $2^{31} - 1$
<code>float</code>	réel, simple précision (IEEE 754)
<code>double</code>	réel, double précision (IEEE 754)

`signed` et `unsigned` rendent le type signé ou non signé (`int` est signé par défaut).

`short`, `long` et `long long` changent la taille d'un `int`. La taille exacte dépend de la machine : `short` est un entier plus court, `long` un entier au moins aussi grand, `long long` un entier encore plus grand. On peut omettre le mot `int`, et combiner ces mots avec `signed` ou `unsigned`.

Ainsi, `long n;` et `unsigned long long n;` sont des déclarations valides. Sur un PC actuel, un `unsigned long long` va de 0 à $2^{64} - 1$. `long long` sert pour un entier trop grand pour un `int`.

On peut utiliser le mot clé `const` au début du type de variable pour indiquer une constante c-à-d une variable dont la valeur reste inchangée le long du programme.

Affecter un `double` à un `int` abandonne la partie fractionnaire : 3.1416 devient 3.

++ et --, avant ou après

`++` ajoute 1, `--` retire 1. L'opérande est une variable. La variable est modifiée dans les deux cas. Ce qui change, c'est la **valeur de l'expression** :

- **après** la variable (`y++`, `y--`) : l'expression vaut la variable **avant** la modification ;
- **avant** la variable (`++y`, `--y`) : l'expression vaut la variable **après** la modification.

Écrit seul, `i++` et `++i` font la même chose : on n'utilise pas la valeur de l'expression. La différence n'apparaît que si cette valeur est lue, par exemple dans une affectation ou un `printf`.

On n'écrit pas `x = --x + x++` : la syntaxe est correcte, le résultat n'est pas défini.

Les conditions

Une comparaison s'écrit `<`, `>`, `<=`, `>=` ou `!=`. L'égalité s'écrit avec `==` accolés, l'affectation avec un seul `=`. En C, faux vaut `0` et vrai vaut n'importe quel entier non nul.

`&&` est le et, `||` le ou, `!` la négation. `&&` et `||` s'évaluent de gauche à droite et s'arrêtent dès que le résultat est connu : dans `n != 0 && m / n > 1`, la division n'a lieu que si `n` n'est pas nul.

`if` choisit une branche.

```
if (expr)
    instr1;
else
    instr2;
```

Si `expr` est vraie, on exécute `instr1`. Sinon, et seulement si le `else` est présent, on exécute `instr2`. Chaque branche peut être un bloc. Un `else` se rattache au `if` le plus proche du même bloc. Trois cas s'écrivent `if / else if / else`. Un `if` suivi de `return` quitte la fonction tout de suite.

`switch` choisit parmi plusieurs cas, pour un entier ou un caractère. Les valeurs des `case` sont des constantes. `break` quitte le `switch` ; sans lui, l'exécution continue au `case` suivant. `default` traite les valeurs qui ne correspondent à aucun `case`.

Les boucles

Les trois boucles répètent une instruction, ou un bloc `{ ... }`.

`while` teste **avant** chaque tour. Si le test est faux dès le début, le corps ne s'exécute pas.

```
while (n >= 2) {
    i++;
    n = n / 2;
}
```

`do ... while` teste **après** le tour. Le corps s'exécute au moins une fois. En général, l'exécution se fera une fois de plus qu'avec une boucle `while` contenant le même gardien de boucle

```
do
    instr;
while (expr);
```

`for` place trois arguments : l'initialisation, le test, et ce qui se fait entre deux tours.

```
for (expr1; expr2; expr3)
    instr;
```

équivalent à

```
{
    expr1;
    while (expr2) {
        instr;
        expr3;
    }
}
```

`expr1` , `expr3` et le corps peuvent être absents. Sans `expr2` , le test est toujours vrai. `expr1` peut être une déclaration : `for (int i = 0; i < n; i++)` .

Si les variables ont déjà une valeur, l'initialisation peut rester vide. On teste, on exécute le corps, **puis** on fait la mise à jour :

```
for (; n - k > 0; n -= k)
    result *= n;
```

printf et scanf

Les deux sont des fonctions de `stdio.h` , pas des mots du langage. Tout programme qui affiche ou lit commence par `#include <stdio.h>` .

`printf` affiche. `scanf` lit au clavier et écrit les valeurs **à l'adresse** des variables : on passe `&n` , pas `n` .

```
printf("Entrez deux entiers : ");
scanf("%d %d", &n, &k); //& donne l'adresse mémoire de la variable
printf("Le résultat est %d \n", result); //%d est utiliser pour afficher
l'entier result et \n permet un passage à la ligne
```

Spécificateur	Type
%d	int
%u	unsigned int
%lf	double
%lld	long long
%.2lf	double affiché avec deux décimales

\n passe à la ligne.